Large Language Model Supply Chain: Open Problems From the Security Perspective

Qiang Hu
Tianjin University
China
qianghu@tju.edu.cn

Xiaofei Xie
Singapore Management University
Singapore
xfxie@smu.edu.sg

Sen Chen*
Nankai University
China
senchen@nankai.edu.cn

Lili Quan
Tianjin University
China
quanlili@tju.edu.cn

Lei Ma
The University of Tokyo & University of Alberta
Japan & Canada
ma.lei@acm.org

ABSTRACT

Large Language Model (LLM) is changing the software development paradigm and has gained huge attention from both academia and industry. Researchers and developers collaboratively explore how to leverage the powerful problem-solving ability of LLMs for specific domain tasks. Due to the wide usage of LLM-based applications, e.g., ChatGPT, multiple works have been proposed to ensure the security of LLM systems. A comprehensive understanding of the entire process of LLM system construction (the LLM supply chain) is crucial, but relevant works are limited. More importantly, the security issues hidden in the LLM SC that could greatly impact the reliable usage of LLMs lack exploration. Most existing works mainly focus on assuring the quality of LLM from the model level; however, security assurance for the entire LLM SC is ignored. In this work, we take the first step to discuss the potential security risks in each component and the integration between components of LLM SC. We summarize 12 security-related risks and provide promising guidance to help build safer LLM systems. We hope our work can facilitate the evolution of artificial general intelligence with secure LLM ecosystems.

CCS CONCEPTS

• **Computing methodologies** → **Machine learning**; • **Software and its engineering** → **Software creation and management**.

KEYWORDS

Large Language Model; Software Supply Chain; Security Assessment

*corresponding author, email: senchen@nankai.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ISSTA Companion '25, June 25–28, 2025, Trondheim, Norway
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1474-0/2025/06
https://doi.org/10.1145/3713081.3731747

ACM Reference Format:
Qiang Hu, Xiaofei Xie, Sen Chen*, Lili Quan, and Lei Ma. 2025. Large Language Model Supply Chain: Open Problems From the Security Perspective. In *34th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA Companion '25), June 25–28, 2025, Trondheim, Norway.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3713081.3731747

1 INTRODUCTION

Large Language Models (LLMs) lead the new era of artificial intelligence (AI). With this success, LLM-driven applications have achieved exciting and even human-better results in multiple domains, such as video generation [15], mathematical competition [18], code generation [17], and autonomous driving [28]. Recently, many big-tech companies have been trying to develop their own LLMs and construct relevant systems for specific tasks and products, pushing researchers and developers to explore more reliable LLM system construction roads.

Similar to conventional software systems, the construction of LLM systems consists of multiple components and participants, e.g., data providers and model developers. The integration of those complex components is called supply chain (SC) from the perspective of software engineering where software (LLM) serves as the core surrounding numerous upstream and downstream participants. More importantly, as LLMs are widely used nowadays, and sometimes in safety- and security-critical situations such as autonomous driving systems, ensuring the reliability of the whole LLM system is important. While the supply chain of conventional software systems has been extensively studied over the past decade, research specifically focused on the supply chain of AI systems, particularly LLMs, remains limited.

Although recent studies explore security-related issues in LLMs, most of them remain focused on the model itself. We analyzed 57 relevant papers from recent surveys [5, 9, 14, 24], and found that 19 of these still concentrate on model-level security, while 24 are exclusively focused on ChatGPT, largely overlooking the broader supply chain. For instance, researchers have shown that adversarial attack techniques can be used to exploit LLMs [7], and LLMs can be easily jailbroken [6]. The full paper list we analyzed can be found on our project site [4]. However, the model is just one component of an LLM system. Even if the model security is ensured, vulnerabilities in other parts of the LLM supply chain, such as third-party dependencies or deployment environments,

can still pose significant security risks and lead to an unreliable LLM system. Therefore, analyzing LLM supply chain security is crucial. While some recent studies [24, 25] explore components of the LLM supply chain, including infrastructure, model lifecycle, and the application ecosystem, their focus remains primarily on the model lifecycle, considering each component separately. The dependencies (i.e., upstream and downstream components) within the LLM supply chain and the associated security risks along the supply chain are still unclear.

To fill this gap, we analyze potential security risks in the LLM supply chain and propose guidelines to mitigate these risks. Specifically, we begin by defining each component of the LLM supply chain and analyzing the dependencies of those components, e.g., from upstream data providers to downstream LLM applications (end users). We then focus on identifying security risks that originate from upstream components, which attackers could exploit to impact downstream participants. For instance, attackers may inject poisoned data into training datasets (upstream), affecting the trained LLMs and, consequently, the deployed LLM-based applications (downstream). In total, we identify 12 security risks related to the LLM supply chain and offer guidance to support the construction of secure LLM systems. We believe that our work can help researchers and developers better understand LLM supply chain security and build more reliable LLM systems.

To summarize, the main contributions of this paper are: 1) we explore security risks considering the integration of dependent components in the LLM SC where we summarize 12 relevant LLM SC risks. The detailed works we analyzed can be found at our project site [4]; 2) we provide promising guidelines to mitigate the risks in developing LLM systems.

2 RISKS IN LARGE LANGUAGE SUPPLY CHAIN

2.1 LLM Supply Chain Extraction

To understand the potential security risks, we first identify the components in the LLM supply chain. Each component could affect not only itself but also the downstream participants. The process of identifying the LLM supply chain involves tracing the potential upstream components that influence or depend on a given component. Specifically, we start with downstream users who may seek an LLM application from an App store or the Internet. The App Store then becomes a potential upstream component. The LLM application in the store has further upstream components, such as the LLM and other software dependencies. By iteratively tracing these dependencies, we continue identifying relevant components until we reach the highest upstream elements where we choose to stop based on the relevance. Finally, we constructed a dependency graph representing the LLM supply chain, as shown in Figure 1.

2.2 LLM Supply Chain

Based on our extracted LLM supply chain, we introduce each component in the chain from the data provider to the end user (from upstream to downstream). In the data preparation phase, data analysts collect data from the data providers and do data cleaning to filter noisy data and features. After that, data labelers (human or labeling machine) assign ground truth to the raw data which will be utilized in the following training and testing processes. Due to the large amount of data we can obtain daily, data selection is an optional process to reduce the data processing effort. In this data construction phase, the reliability of data selection, data cleaning, and automatic data labeling tools highly affects all the remaining downstream components.

The downstream phase of data preparation is model construction. Roughly speaking, there are three ways to prepare a LLM, 1) prepare it from scratch, 2) download it from the open-source model hubs and directly use it, and 3) download it and then fine-tune it for specific domain tasks. When preparing LLMs from scratch, developers utilize AI frameworks and third-party libraries to write the training program, and then train the model with pre-training data provided by the data analysts. For the second way, developers directly employ the downloaded LLMs for the target task. For the last way, the downloaded LLMs are further fine-tuned using fine-tuning datasets to fit a new data distribution. After the model training, test reports are generated based on the test dataset, which will be provided to the application developers along with the trained LLMs. In this model construction phase, concerns could arise due to the vulnerabilities in the program, the model training process, the conflicts between pre-training and fine-tuning, and malicious hidden in model hubs.

After application developers obtain the model, based on specific requirements such as the application size limitation, they will decide whether to utilize model optimization techniques to reduce the model size. The LLMs are then integrated with other software, including additional AI models, to form the final LLM application package, which is then uploaded to the application store. Finally, the downstream users download the application for real-world usage. In this application preparation phase, risks are potentially hidden in other software and the model optimization process.

2.3 LLM Supply Chain Risk Analysis

Based on the LLM supply chain, we discuss potential security risks by identifying attack paths that originate from upstream components and target downstream users. These attack paths allow adversaries to compromise users by executing malicious operations at upstream points. In total, we identify and summarize 12 potential risks across the entire LLM SC.

Risks in Data Construction Phase. First, since data is the key driver for LLM, security assessment of datasets is crucial as training data will affect the whole LLM SC. Treating the data provider as the upstream supplier, the downstream participant should be the data analyst. Existing works [5] summarized the risks in the training data from different perspectives, such as backdoor attacks, poisoning attacks, privacy leakage, unauthorized disclosure and privacy concerns, bias, toxicity, and ethics. Some risks directly impact downstream components, such as poisoning attacks that degrade the performance of trained LLMs and inject backdoors, compromising model predictions and user security. From the perspective of the LLM supply chain, several security risks need to be addressed:

Risk1: Risks in data selection process. Utilizing the whole collected dataset to build LLM systems is almost impossible due to the large amount of available data. Data selection is a technique of filtering high-quality data for training or testing and will affect all the remaining components in the LLM SC. Recent research [13, 22] demonstrated that existing automatic data selection methods have

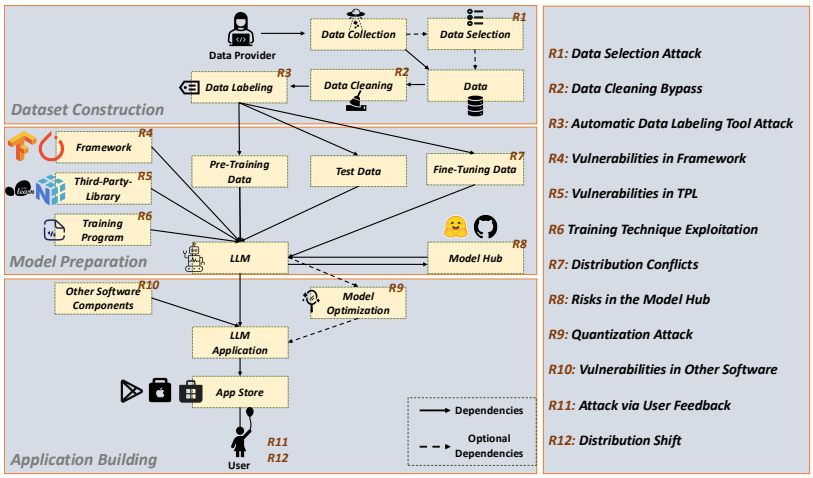


Figure 1: Security risks in LLM supply chain.

limitations such as only considering uncertain data which can be exploited by attackers to inject malicious into the collected dataset. For example, attackers can inject backdoors into the data that have high uncertainty scores to multiple common LLMs, and lead the data selection methods to select certain data from the dataset.

Risk2: Risks in data cleaning process. After collecting datasets, data analysts will employ data cleaning techniques to remove redundant and noisy data (and features). However, potential risks are hidden in this process, 1) existing data cleaning techniques are relatively simple, such as rule-based error sample detection [19]. It is easy to bypass such techniques by constraint-ware adversarial attacks [21] and introduce malicious to the datasets. 2) More generally, even though data scientists can guarantee the security of cleaned data for the downstream model training by collaborating with model developers, it is unclear whether the cleaned data still contains risks that affect further LLM SC components, such as model maintenance and model compression.

Risk3: Risks in data labeling process. Normally, given a domain task, a useful data sample requires the raw data and the corresponding ground truth. Labeling a large amount of raw data is challenging and may naturally introduce label errors. For example, even for simple image classification tasks, incorrect labels exist in the dataset after carefully manually labeling [20]. This mislabeling issue can mislead the model training and testing, and harm the performance of LLMs. Besides, multiple automatic data labeling techniques and frameworks have been proposed recently [23]. The other risk is such tools also rely on deep learning models and can be fooled to generate incorrect labels by adversarial attacks.

Risks in Model Preparation Phase. In addition to the risks in the data, there are also many security issues hidden in the model preparation phase. For example, the vulnerabilities in the training program, the improper use of training methods, and the different distribution between the pre-training and fine-tuning datasets can affect the produced LLMs and further introduce different reliability issues to the downstream components.

Risk4 and Risk5: Vulnerabilities hidden in the AI framework and third-party-libraries. Even though many works have attempted to reveal the vulnerabilities in the DL frameworks and other third-party libraries to ensure safe LLM development, they mainly focus on the influence of such vulnerabilities on the software implementation and the AI models. Even worse, some vulnerabilities could arise in later downstream components in the LLM SC, for example, existing works [10] found that implementation errors in the framework can introduce serious prediction errors of AI applications on the website and cause unreliable outputs. Therefore, in addition to the software and models, analyzing the potential influence of vulnerabilities on the following LLM SC is necessary. Besides, recent works demonstrated that attackers can modify APIs in the framework and inject backdoors into the binary model files to assess information from users [3]. Risk assurance of frameworks and TPLs is crucial for securing the usage of LLM applications for users.

Risk6: Risks introduced in training techniques. Researchers proposed multiple training techniques such as active learning and adversarial training to prepare models for different purposes. Recent works reveal that such training techniques will introduce different reliability issues to the compressed models, e.g., active learning reduces the adversarial robustness of models after quantization [12]. Therefore, it is needed to understand how different training techniques affect the following SC components. Besides, training techniques can be also exploited by attackers. For instance, similar to the issues mentioned in the data selection process, it is possible to attack active learning methods to force models to select vulnerable data for training.

Risk7: Distribution conflicts between pertaining datasets and fine-tuning datasets. As shown in Figure 1, LLMs can be prepared by fine-tuning the pre-trained model given specific domain datasets. Unfortunately, as shown in some recent works [11, 16], pertaining datasets and fine-tuning datasets can sometimes have distribution conflicts and lead to the forgetting problem where the fine-tuned LLMs will lose pre-trained knowledge. Therefore, in addition to the performance degradation of the LLMs on the original domain, risks arise that attackers can utilize fine-tuned datasets to break the defense techniques in the pre-trained LLMs (e.g., adversarial-trained LLMs) and find backdoors to attack the LLM systems.

Risk8: Risks in the model hub. In the LLM fine-tuning process, developers usually download pre-trained LLMs from an open-source

model hub such as Hugging Face [2]. Even though such model hubs have a security scan procedure to check the reliability of uploaded LLMs, current work [29] demonstrated that these scanning techniques can be easily bypassed. Therefore, the downloaded LLMs might contain different security issues and need to be carefully checked before the fine-tuning process.

Risks in Application Building Phase. Deploying deep learning models to real-world applications is the final goal. A recent report [1] shows that most safety issues (24%) happen at the system level. Existing works pay more attention to the model-level evaluation for security assurance but lack a study of LLM-based applications. Since the downstream participants of LLM-based applications are users, the security risks will cause practical damage.

Risk9: Risks introduced by model optimization. Recent works [8] revealed that attackers can utilize model compression techniques to hide backdoors to the LLM where the original LLM is secure but the compressed LLM is vulnerable. The risks can be introduced by the model optimization process. It is necessary to carefully choose the model compression techniques and avoid potential attacks.

Risk10: Vulnerability in other software components. In addition to the model, LLM applications could contain other software components. For example, a LLM-driven autonomous driving system could contain multiple components such as the perception module, the planning module, and the control module. Software components could affect each other and influence the final output of the system, e.g., the unreliable images recorded by the camera could mislead the decision of the following components in the LLM system. Therefore, it is necessary to analyze the influence and dependencies of such software components in the LLM application security.

Risk11: Security risks via user feedback. LLM providers will utilize user feedback (prompts and answers) to update models for domain adaptation. In this situation, the LLM provider could be the downstream participant while users are the upstream suppliers. Unfortunately, there is a concern that the user feedback could also introduce security-critical samples to the fine-tuning dataset which can be exploited by attackers, and then harm the reliability of LLMs.

Risk12: Risks due to the unknown tasks and data distribution. Every evaluation is based on a prepared test dataset that covers a certain distribution domain, however, it is unable to estimate the usage environment of the LLM-based applications. Thus, it is impossible to guarantee that the pre-defined risk assessments can always work. The security issues of real-world LLM applications are the most difficult to handle.

3 RISK MITIGATION DISCUSSION

Based on our analysis, we provide guidelines for future research on secure LLM SC from data, model, and application three perspectives.

Security assurance in the data construction. Data collectors should carefully use automatic data selection and data cleaning methods to collect datasets, as they can be easily fooled. Then, the upstream supplier is recommended to provide some data distribution information to the downstream users, such as the top-N token frequency distribution. In this way, developers can control the domain distribution in the following model testing and fine-tuning processes to report more precise and reliable evaluation results, and prepare fine-tuned models without harming the original distribution.

Security assurance in the model preparation. LLM preparation is the key component in the LLM SC. In the training phase, fully-supervised active learning should be carefully employed as it could reduce the robustness of the model and can be attacked. Semi-supervised active learning with data augmentation is a better choice that can mitigate attacks with data modification. Considering the downstream action model deployment of LLMs, developers need to think about whether it is necessary to use quantization-aware (or other compression-aware) training techniques. Besides, it is useful to highlight data samples that are uncertain to the LLM before compression as the compressed LLM could produce different outputs as shown in previous works [27].

Security assurance in the application development The upstream participant of applications is user. Thus, security assurance of LLM-based applications is a crucial and emerging task. First, every LLM-based application needs a test report on the targeted domains that covers diverse evaluation metrics (especially security risk evaluation) to guide its reliable usage. Besides, as mentioned in Section 2.3, multiple types of attacks can be injected into applications and some can bypass existing defense techniques such as ModelScan. Therefore, application maintainers must design stronger scan techniques to check uploaded applications, specifically focusing on embedded operators (APIs) that can assess users' devices.

Quality assurance of a single component in the LLM SC is not enough to ensure the reliability of the final LLM systems. In the real-world LLM SC, upstream suppliers can directly affect their connected downstream participants and indirectly related participants in different aspects. It is essential to design metrics and criteria to measure the influence of one component on another for risk assurance of the whole SC.

4 FUTURE PLAN

In the future, we plan to: 1) make a comprehensive survey on LLM SC security risks; 2) design metrics and criteria to measure the influence caused by security issues between different components in the LLM SC and modeling the influence chain, e.g., design the influence chain as: $C_0 \overset{(R_0,\alpha_0)}{\longrightarrow} C_1 \overset{(R_1,\alpha_1)}{\longrightarrow} \dots \overset{(R_{n-1},\alpha_{n-1})}{\longrightarrow} C_n$, where C is the component in the SC, n is the number of components, R is the risk type, and α is the severity of the risk; 3) propose techniques for the security assurance of LLM SC, for example, design multi-objective guided data selection methods for targeted instruction tuning [26] to efficiently train LLMs with better performance while avoiding potential attacks on the data selection process.

5 CONCLUSION

In this paper, we have initiated a discussion about security risks in the large language model supply chain. Different from existing works that mainly focused on studying a single component (especially the model) in the SC, we explored the potential risks that lie in the integration of the SC from upstream suppliers to downstream users. In total, we summarized 12 risks that have gained limited attention by current research and provided promising guidance for both researchers and developers to help construct more reliable LLM systems. We believe ensuring the security of LLM SC is crucial and our work can facilitate the understanding of the risks.

REFERENCES

[1] 2024. AI Risk Repository. https://airisk.mit.edu/

[2] 2024. Hugging Face. https://huggingface.co/

[3] 2024. ModelScan. https://github.com/protectai/modelscan

[4] 2025. LLMSC: collected papers. https://sites.google.com/view/llmscsec,

[5] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2024. A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology* 15, 3 (2024), 1–45.

[6] Gelei Deng, Yi Liu, Yuekang Li, Kailong Wang, Ying Zhang, Zefeng Li, Haoyu Wang, Tianwei Zhang, and Yang Liu. 2023. Jailbreaker: Automated jailbreak across multiple large language model chatbots. *arXiv preprint arXiv:2307.08715* (2023).

[7] Yinpeng Dong, Huanran Chen, Jiawei Chen, Zhengwei Fang, Xiao Yang, Yichi Zhang, Yu Tian, Hang Su, and Jun Zhu. 2023. How Robust is Google's Bard to Adversarial Image Attacks? *arXiv preprint arXiv:2309.11751* (2023).

[8] Kazuki Egashira, Mark Vero, Robin Staab, Jingxuan He, and Martin Vechev. 2024. Exploiting LLM Quantization. *arXiv preprint arXiv:2405.18137* (2024).

[9] Mingyuan Fan, Cen Chen, Chengyu Wang, and Jun Huang. 2023. On the trustworthiness landscape of state-of-the-art generative models: A comprehensive survey. *arXiv preprint arXiv:2307.16680* (2023).

[10] Qianyu Guo, Sen Chen, Xiaofei Xie, Lei Ma, Qiang Hu, Hongtao Liu, Yang Liu, Jianjun Zhao, and Xiaohong Li. 2019. An empirical study towards characterizing deep learning development and deployment across different frameworks and platforms. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 810–822.

[11] Qiang Hu, Yuejun Guo, Maxime Cordy, Xiaofei Xie, Lei Ma, Mike Papadakis, and Yves Le Traon. 2022. An empirical study on data distribution-aware test selection for deep learning enhancement. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 4 (2022), 1–30.

[12] Qiang Hu, Yuejun Guo, Maxime Cordy, Xiaofei Xie, Wei Ma, Mike Papadakis, and Yves Le Traon. 2021. Towards exploring the limitations of active learning: An empirical study. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 917–929.

[13] Qiang Hu, Yuejun Guo, Xiaofei Xie, Maxime Cordy, Wei Ma, Mike Papadakis, and Yves Le Traon. 2023. Evaluating the robustness of test selection methods for deep neural networks. *arXiv preprint arXiv:2308.01314* (2023).

[14] Xiaowei Huang, Wenjie Ruan, Wei Huang, Gaojie Jin, Yi Dong, Changshun Wu, Saddek Bensalem, Ronghui Mu, Yi Qi, Xingyu Zhao, et al. 2024. A survey of safety and trustworthiness of large language models through the lens of verification and validation. *Artificial Intelligence Review* 57, 7 (2024), 175.

[15] Chunyuan Li, Cliff Wong, Sheng Zhang, Naoto Usuyama, Haotian Liu, Jianwei Yang, Tristan Naumann, Hoifung Poon, and Jianfeng Gao. 2024. Llava-med: Training a large language-and-vision assistant for biomedicine in one day. *Advances in Neural Information Processing Systems* 36 (2024).

[16] Zhizhong Li and Derek Hoiem. 2017. Learning without forgetting. *IEEE transactions on pattern analysis and machine intelligence* 40, 12 (2017), 2935–2947.

[17] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2024. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems* 36 (2024).

[18] Yujun Mao, Yoon Kim, and Yilun Zhou. 2024. CHAMP: A Competition-level Dataset for Fine-Grained Analyses of LLMs' Mathematical Reasoning Capabilities. In *Findings of the Association for Computational Linguistics ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand and virtual meeting, 13256–13274. https://doi.org/10.18653/v1/2024.findings-acl.785

[19] Zhengjie Miao, Yuliang Li, and Xiaolan Wang. 2021. Rotom: A meta-learned data augmentation framework for entity matching, data cleaning, text classification, and beyond. In *Proceedings of the 2021 International Conference on Management of Data*. 1303–1316.

[20] Curtis G Northcutt, Anish Athalye, and Jonas Mueller. 2021. Pervasive label errors in test sets destabilize machine learning benchmarks. *arXiv preprint arXiv:2103.14749* (2021).

[21] Thibault Simonetto, Salijona Dyrmishi, Salah Ghamizi, Maxime Cordy, and Yves Le Traon. 2021. A unified framework for adversarial attack and defense in constrained feature space. *arXiv preprint arXiv:2112.01156* (2021).

[22] Weifeng Sun, Meng Yan, Zhongxin Liu, and David Lo. 2023. Robust Test Selection for Deep Neural Networks. *IEEE Transactions on Software Engineering* 49, 12 (2023), 5250–5278.

[23] Shuohang Wang, Yang Liu, Yichong Xu, Chenguang Zhu, and Michael Zeng. 2021. Want to reduce labeling cost? GPT-3 can help. *arXiv preprint arXiv:2108.13487* (2021).

[24] Shenao Wang, Yanjie Zhao, Xinyi Hou, and Haoyu Wang. 2024. Large language model supply chain: A research agenda. *arXiv preprint arXiv:2404.12736* (2024).

[25] Shenao Wang, Yanjie Zhao, Zhao Liu, Quanchen Zou, and Haoyu Wang. 2025. SoK: Understanding Vulnerabilities in the Large Language Model Supply Chain. *arXiv preprint arXiv:2502.12497* (2025).

[26] Mengzhou Xia, Sadhika Malladi, Suchin Gururangan, Sanjeev Arora, and Danqi Chen. 2024. Less: Selecting influential data for targeted instruction tuning. *arXiv preprint arXiv:2402.04333* (2024).

[27] Xiaofei Xie, Lei Ma, Haijun Wang, Yuekang Li, Yang Liu, and Xiaohong Li. 2019. Diffchaser: Detecting disagreements for deep neural networks. International Joint Conferences on Artificial Intelligence Organization.

[28] Zhenhua Xu, Yujia Zhang, Enze Xie, Zhen Zhao, Yong Guo, Kwan-Yee K Wong, Zhenguo Li, and Hengshuang Zhao. 2024. Drivegpt4: Interpretable end-to-end autonomous driving via large language model. *IEEE Robotics and Automation Letters* (2024).

[29] Jian Zhao, Shenao Wang, Yanjie Zhao, Xinyi Hou, Kailong Wang, Peiming Gao, Yuanchao Zhang, Chen Wei, and Haoyu Wang. 2024. Models Are Codes: Towards Measuring Malicious Code Poisoning Attacks on Pre-trained Model Hubs. *arXiv preprint arXiv:2409.09368* (2024).